Linear Search : ✓ Both iterative and recursive

Divide and Conquer:

Divide → Break to sub problem

Conquer → Solve

Combine → solution to sub problem

Binary Search: * Sorted must

* Compare with middle element

* Find peak (Reach one of them at least)

Type of order: (1) Strictly increasing : 2, 4, 5, 7, 8 [No repeat]

(2) Strictly decreasing : 10, 6, 4, 3, 1 [No repeat]

(3) Non-increasing : 10, 6, 6, 4, 3, 3, 1, 1 [Equal possible]

(4) Non-decreasing : 1, 2, 2, 3, 4, 4, 4, 5, 5 ["]

arr[mid] < [mid+1]
left = mid+1

l		mid		r
7	12	34	58	87

Find ~~34~~ 60

* Find max from bitonic

(▭ ▭)
Ascend Descend

$O(\log_2 n)$

```
boolean binarySearch(A[], l, r, target){
    while(l<=r){
        mid=(l+r)/2;          — O(1)
        if(A[mid]==target) return true
        else{
            if(target<A[mid]) r=mid-1
            else  l=m+1
        }
    }
    ~~return false~~;
}
```

Comparator 1

$1 * \log_2 n$

$\log_3 n = \log_2 n \times \log_3 2$

Ternary Search:

Find 31

3	5	7	12	15	23	27	31	45	56	70	98
Left			Mid1				Mid2				Right

```
boolean TernarySearch (A[], l, r, item){
    while ( l <= r) {
        mid1 = l + (r-l)/3;
        mid2 = r - (r-l)/3;
        if (A[mid1] == item || A[mid2] == item) return true;
        else {
            if (item < A[mid1]) r = mid1 - 1;
            else if (item > A[mid2]) l = mid2 + 1;
            else l = mid1 + 1; r = mid2 - 1;
        }
    }
```

[Combine]
Comparison operator: 2

$2 * \log_3 n$

$2 * \log_2 n * \log_3 2$

$1.26 * \log_2 n$

[computation cost of fn]

Time complexity:

$n/3 \wedge (\text{step no.})$

The last line can be written as $1 = n/3 \wedge K$

$K = \log_3 n$

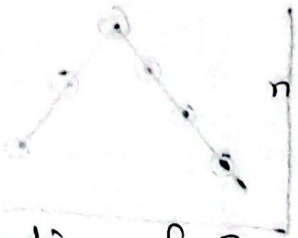
$O(\log_2 n) \leftarrow O(0.63 * \log_2 n) \leftarrow \boxed{O(\log_3 n)}$

$\log_3 2$

Order Notation same

Quaternary Search : * N-ary Search

O(1) → যেগুলো check করতে গেলে linear হয়ে যাবে



$n * \log_n$

Application of Binary Search :

1. Finding an element

3	4	5	8	11	21	80
T	T	T	T	T	F	F

(Key = 21)

Property / condition

Key > any element : the array

```
while (l<=r)
    mid = (l+r)/2 ;
    if (Key>a[mid])  l = mid+1
    else{ ans = mid
          r = mid-1
    }
```

2.

1	2	3	4	5	6	7	8	9	10	11	12	13
3	4	5	5	6	6	6	6	6	8	10	12	13

Key = 6 (leftmost)

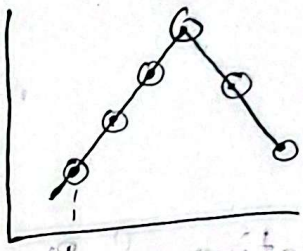
```
ans = -1
while (l<=r)
    mid = (l+r)/2
    if(Key==a[mid])  ans= mid , r=mid-1
    else if (Key>a[mid])  l=mid+1
    else  r=mid-1
```

// Key<a[mid]

3. Right most occurance

4.

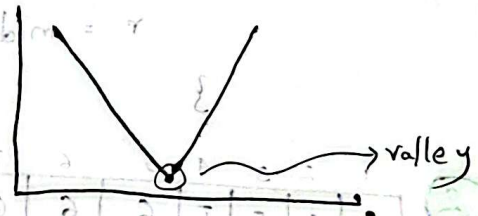
1	2	3	4	5	6	7	8
3	5	7	8	10	7	5	2
T	T	T	T	T	F	F	F



Unimodal graph

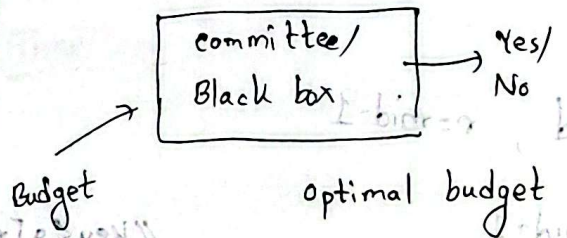
```
l = 1
r = n
ans = -1
while (l <= r)
    mid = (l+r)/2
    if (a[mid] > a[mid-1])   ans = mid;  l = mid+1
    else    r = mid-1
```

5.



Unimodal graph

6. Binary Search on Answer (Budget Plan)



optimal budget

7K	8K	9K	10k	11K	12K	13K
N	N	N	Y	Y	Y	Y

```
l = 0tk
r = 1 lac tk
while (l <= r)
    mid = (l+r)/2
    if (a[mid] == T)   ans = mid
                       right = mid-1
    else  mid+1
```